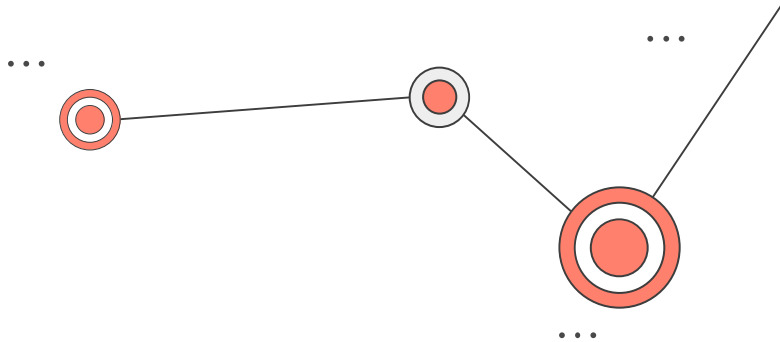
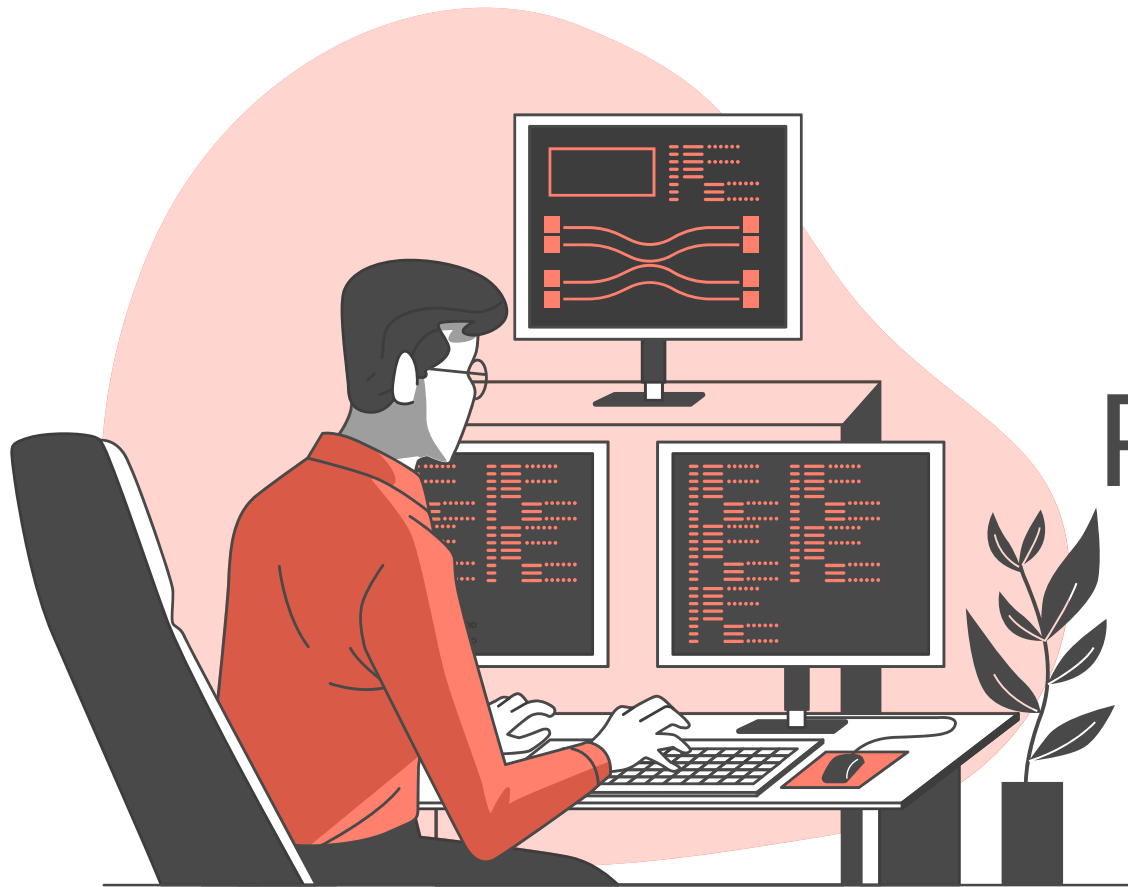
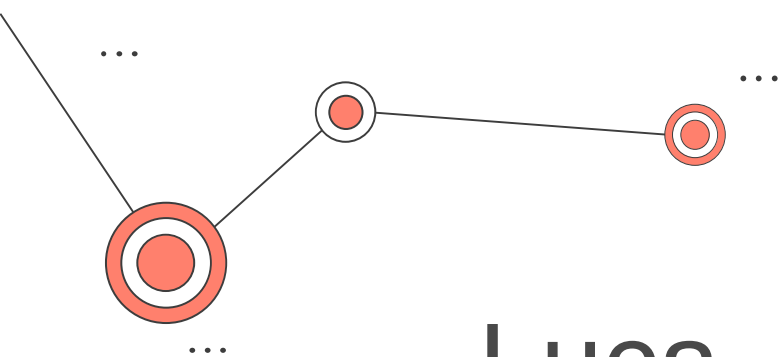




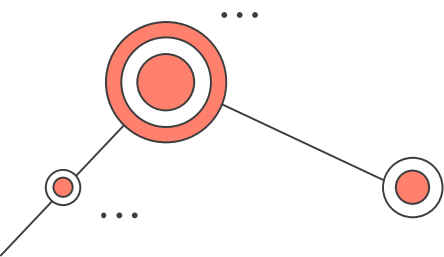
Rust in Production

Ana Calleja Ramón
Adriana Garcia Suarez
Luisa Rodríguez Torres
Emre Sen



Luca Palmieri

Zero to Production in Rust





...

What is "Production" in Rust?



...

Same goals

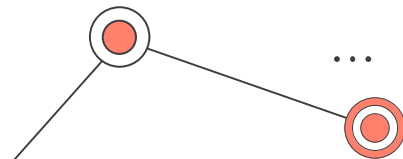
Know when it fails,
restore it, monitor
without print

One difference

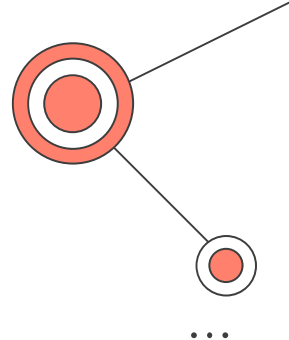
How you achieve
these goals using the
Rust ecosystem's
tools

Guide

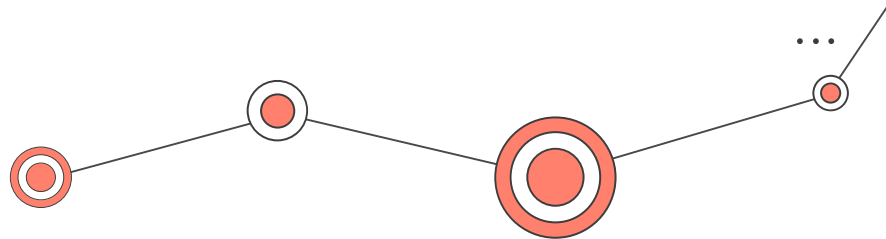
Luca Palmieri, author
of "Zero to Production
in Rust"

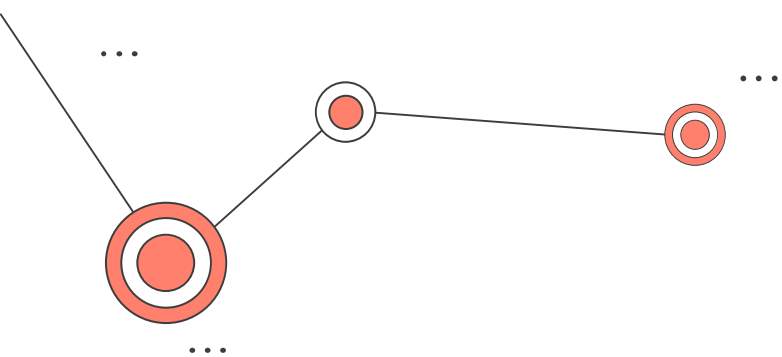


A Key Advantage: The Binary

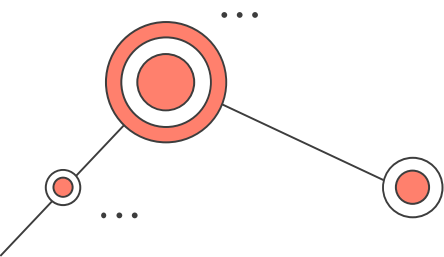


- Rust compiles to a self-contained binary.
- Deployment: Ship the binary or a minimal Docker container.
- Lightweight images leads to faster cold starts in serverless.
- Compared to: Python, JavaScript, C# (which need a runtime).





The Fundamentals of Rust



- Ownership & Borrowing
- The Type System
- Error Handling
- Fearless Concurrency
- Zero-Cost Abstractions
- Pattern Matching
- Traits & Generics

Rust characteristic

S
What makes Rust unique?

Profiles

debug

fast compile, slower
runtime

release

slower compile,
optimized performance
& smaller size

Ecosystem

Crates

unit of compilation

Incremental compilation

speeds up inner
development loop

Minimal standard library

relies on external
crates for features

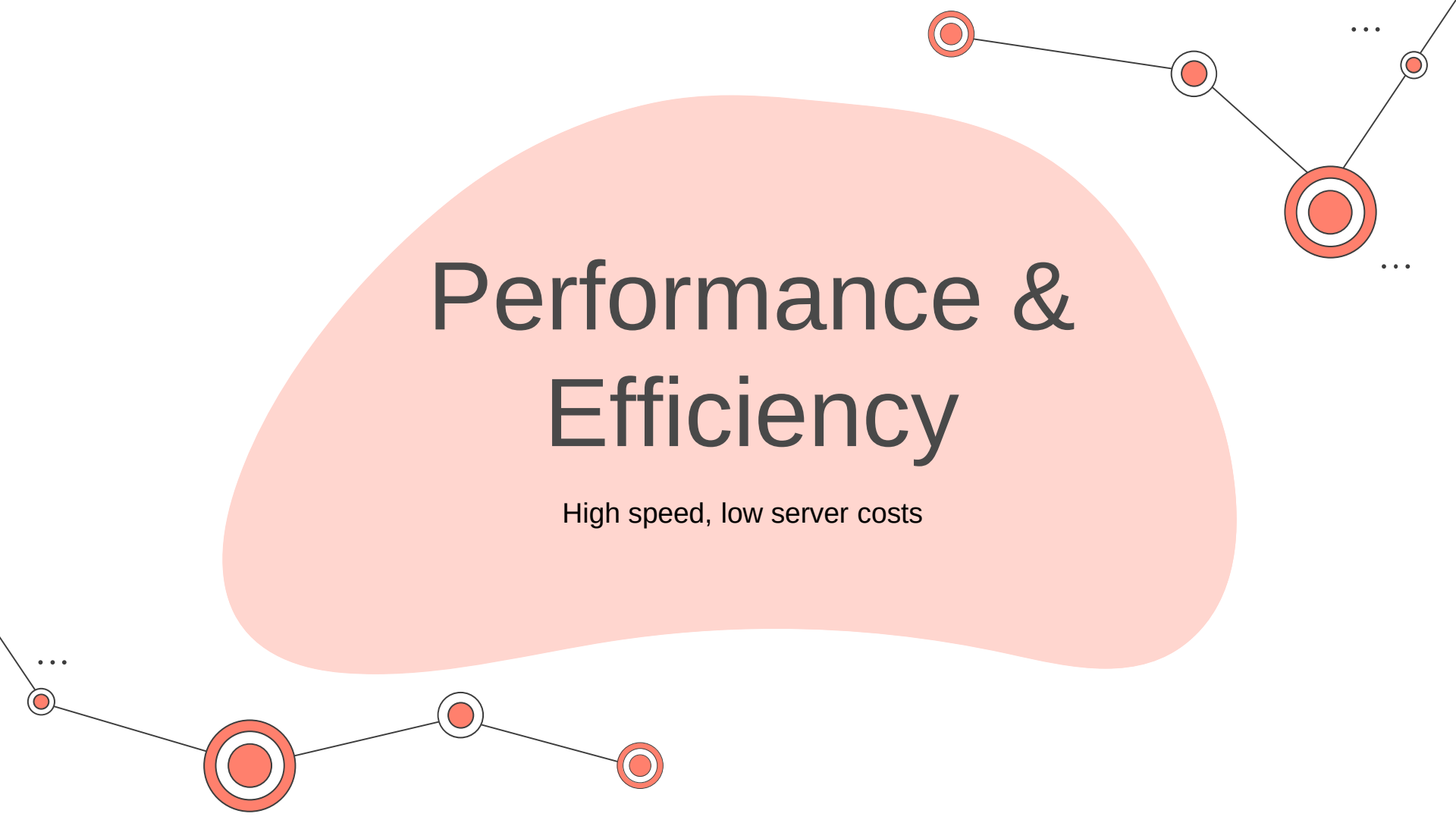
Optimizations

Link-Time Optimization (LTO)

Optimizes across crates
for runtime speed.

Profile-Guided Optimization (PGO)

Compiler uses runtime
usage data to optimize
hot paths.



Performance & Efficiency

High speed, low server costs

Security in Rust

Safe Rust

No garbage collector, but strict memory safety. Protects against buffer overflows and out-of-bounds access automatically. Focus on business logic, not memory leaks.

Supply Chain Security

Minimal standard library means relying on external crates. Must carefully audit dependency trees for malicious code or undetected vulnerabilities..



Key Performance Metrics



High RPS

Efficient async model
CPU multiplexing

Tiny Footprints

Immediate deallocation
~40-50 MB idle memory
Low resource usage

Consistent Latency

Every user gets the
same fast experience
Predictable UX
Uniform performance

Performance & Energy



No Garbage Collector

No pauses to collect garbage
Prevents sudden spikes



Auto-Scaling

Resource usage is predictable
Easy to scale



Cost-Effective

Same traffic lower cost
Tiny cloud compute bills



Green Energy

Less CPU usage
Smaller carbon footprint

Telemetry

The Tracing Tool

Standard logging library
Minimal runtime overhead

Structured Logging

Uses spans
Captures duration & location

Open Telemetry

Standard Interface
Easy export to monitoring tools

Error Handling

What errors can we get?
How to solve them?

Traditional Error Handling

Exceptions

In Java, C#, Python

Errors thrown as exceptions
Travel through the code
Caught using try/catch

Problems ?

- Non-local error handling
- Easy to forget catching errors or updating them
- Lead to bugs

Rust Approach: Result Type



Result Type in Rust

*/*The Result type in Rust is used for functions that can return either a success (Ok) or an error (Err). Handling Result ensures robust error management in Rust programs.*/*

```
fn main() {  
    // Example 1: Handling Ok and Err  
    match divide(10, 0, 2.0) {  
        Ok(result) => println!("Result: {}", result),  
        Err(e) => println!("Error: {}", e),  
    }  
}
```

```
    // Example 2: Handling an error case  
    match divide(10, 0, 0.0) {  
        Ok(result) => println!("Result: {}", result),  
        Err(e) => println!("Error: {}", e),  
    }  
}
```

```
    // Function that returns a Result  
    fn divide(a: f64, b: f64) -> Result<f64, String> {  
        if b == 0.0 {  
            Err(String::from("Cannot divide by zero"))  
        } else {  
            Ok(a / b)  
        }  
    }  
}
```

Two variants:

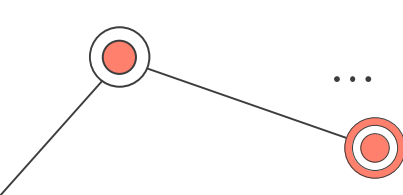
- Ok → returns the data if successful
- Error → returns information about the failure

Advantages

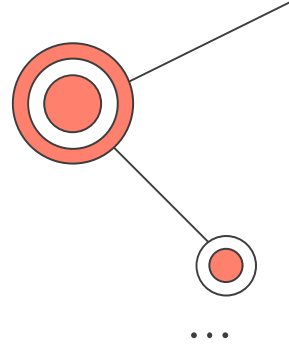
- Safer
- Allows to have type errors
- Forced to handle the error

Pattern Matching:

Specify what happen in both the success and error cases.



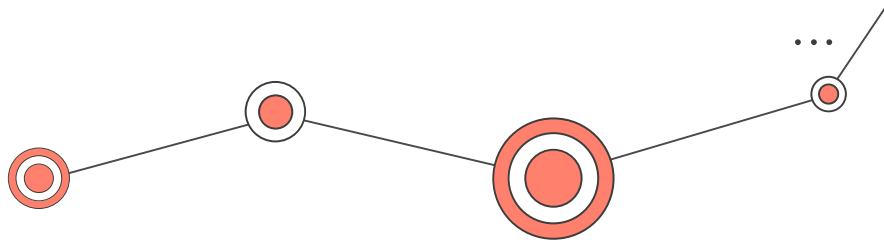
Other ways of handling errors

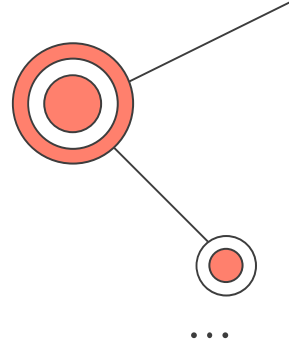
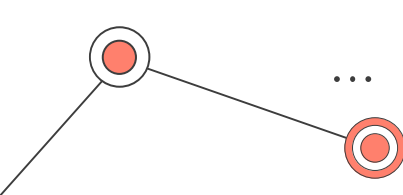


Question mark operator '?' :

- Allows to propagate the error
- Similar to exceptions
- Automatically as long as the two error types are compatible

Panic

- Equivalent to crashing the program
 - Mainly use in development process
- 

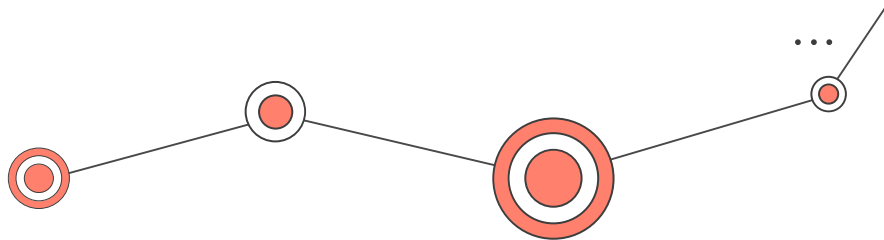


Recommendations

Tools to improve code quality

- Clippy: Detects problematic code patterns

Test coverage tools

- Use LLVM code
- 



Thanks!

Do you have any questions?